Once-Only

One of the principles that the PCRE engine works on is *backtracking*, especially when quantifiers are used. Consider an expression with a greedy matching character class – such as `[^\d]+\s`, which matches one or more non-digits followed by a single space – the engine would match as many non-digit characters as possible before moving to the next byte in the sample. If the next byte does not match, then the engine back-tracks and tries a sub-set of the sample before trying again, and again, and again. Let's say the sample contains `'abcdef123'`, the engine would match on `abcdef`, fail to match on the `1`, and backtracks to `bcdef`, then `cdef`. Eventually it will give up, but not before consuming CPU resources in the process.

It is possible to instruct the PCRE engine to give up on non-matches in such expressions after a single greedy match. We know that `'abcdef123'` will never match `[^\d]+\s`, but PCRE needs to backtrack multiple times before it knows for sure it can give up. Such things can give rise to serious performance issues for complex expressions or large samples, but by enclosing the quantified character class in a special sub-pattern syntax, the PCRE engine will change its behavior. The *once-only* sub-pattern syntax is `(?>...)`. The modified form of our expression is now `(?>[^\d]+)\s)`, and this modified form prevents the backtracking that would otherwise occur. PHP doesn't have any native way to capture performance statistics from the PCRE engine; there's no PHP I can produce that measures the difference in the number of internal backtracking calls that would be made.

There is, however, an abundance of regex tools on the internet. Regular expressions constitute an entire branch of programming expertise in their own right, after all. Here are two screenshots from `https://regex101.com`, which demonstrate the difference in "cycles" when a once-only sub-pattern is used (fewer cycles means fewer backtracking calls were made).